

DL Practical 3

Practical 3 – Fashion MNIST Classification using CNN

This practical is about:

Training a Convolutional Neural Network (CNN) to recognize and classify clothing images into categories like shoes, shirts, bags, dresses, etc.

What Are We Exactly Doing in Practical 3?

We are teaching the computer to:

- Look at clothing images
- Understand image patterns
- Predict which clothing category the image belongs to

This is:

Image Classification

using:

Convolutional Neural Network (CNN)

Dataset Used

Fashion-MNIST dataset contains:

- 70,000 grayscale images
- Image size = 28×28 pixels
- 10 clothing categories

Examples:

- T-shirt
 - Trouser
 - Bag
 - Sneaker
 - Dress
 - Sandal
-

Why CNN is Used?

CNN is specially designed for image processing.

Because CNN can:

- Detect edges
- Detect textures
- Detect shapes
- Learn spatial patterns automatically

Normal ANN/DNN is not very efficient for images.

Real Goal of the Practical

Input:

- Clothing image

Output:

- Predicted clothing category

Example:

Image	Prediction
Shoe image	Sneaker
Bag image	Bag
Dress image	Dress

Full Algorithm Explanation

Step 1: Import Required Libraries

Libraries used:

- numpy
- matplotlib
- keras
- tensorflow

Purpose:

- Image processing
 - Deep learning
 - Visualization
-

Step 2: Load Fashion-MNIST Dataset

Dataset loaded using:

```
fashion_mnist.load_data()
```

Dataset contains:

- Training images
 - Testing images
 - Labels
-

Step 3: Understand Dataset Shape

Images shape:

```
(60000, 28, 28)
```

Meaning:

- 60,000 images
 - Each image = 28×28 pixels
-

Step 4: Display Sample Images

The code plots clothing images.

Purpose:

- Visualize dataset
 - Verify images loaded correctly
-

Step 5: Reshape Images

CNN expects image format:

```
(28,28,1)
```

Where:

- 28×28 = image size
 - 1 = grayscale channel
-

Why Reshaping is Needed?

CNN layers require:

- height
- width
- channels

So reshaping makes images compatible with CNN.

Step 6: Normalize Pixel Values

Original pixel values:

- 0 to 255

Converted into:

- 0 to 1

Using:

```
train_norm=trainX/255.0
```

Why Normalize?

Normalization:

- Improves training speed
- Helps stable learning
- Prevents very large values

Step 7: One-Hot Encoding Labels

Labels:

- 0 to 9

Converted into binary vectors.

Example:

Class 3:

```
[0001000000]
```

Why One-Hot Encoding?

Neural networks work better with vector outputs.

Step 8: Build CNN Model

This is the MOST IMPORTANT step.

The model contains:

```
Conv2D()  
MaxPooling2D()  
Flatten()  
Dense()
```

Understanding CNN Layers

1. Convolution Layer

```
Conv2D(32, (3,3))
```

Purpose:

- Detect image features

Like:

- edges
- curves
- textures

How Convolution Works

A small filter/kernel moves across image.

It extracts patterns.

Example:

- shoe edge
- shirt sleeve
- bag shape

2. ReLU Activation

Used after convolution.

$$\text{ReLU}(x) = \max(0, x)$$

Purpose:

- Introduces nonlinearity
 - Helps model learn complex patterns
-

3. Pooling Layer

```
MaxPooling2D()
```

Purpose:

- Reduce image size
 - Reduce computation
 - Keep important features
-

Max Pooling

Selects maximum value from region.

Example:

From:

1	5
2	3

Max pooling selects:

- 5
-

Why Pooling is Important?

Pooling:

- Speeds training
 - Reduces overfitting
 - Makes model efficient
-

4. Flatten Layer

Converts:

- 2D feature maps
into
- 1D vector

Before sending to dense layer.

5. Dense (Fully Connected) Layer

Purpose:

- Final classification

Predicts:

- which clothing class image belongs to
-

Output Layer

Final layer contains:

- 10 neurons

Because:

- 10 clothing categories exist.
-

Step 9: Compile Model

The code uses:

```
loss='categorical_crossentropy'  
optimizer='adam'
```

```
metrics=['accuracy']
```

Categorical Crossentropy

Used because:

- Multiple classes exist (10 classes)

Measures classification error.

Adam Optimizer

Purpose:

- Update weights
- Minimize loss
- Improve learning speed

Advanced version of Gradient Descent.

Step 10: Train Model

The model learns using:

```
model.fit()
```

During training:

1. Images passed through CNN
2. Features extracted
3. Predictions made
4. Error calculated
5. Weights updated

This repeats for many epochs.

Epoch Meaning

1 Epoch =

One complete pass through all training images.

Step 11: Evaluate Model

Testing data used to:

- check accuracy
- evaluate performance

The practical achieves:

- around 90–95% accuracy
-

What CNN Learns Internally

Early layers learn:

- edges
- lines

Middle layers learn:

- shapes
- patterns

Deep layers learn:

- full clothing structure

Example:

- shoe sole
 - shirt collar
 - bag handle
-

Very Important Viva Answer

"What are we exactly doing in this practical?"

Strong answer:

In this practical, we are implementing a Convolutional Neural Network on the Fashion-MNIST dataset for image classification. The CNN automatically extracts features from clothing images using convolution and pooling layers and classifies them into one of the 10 fashion categories.

Short Algorithm for Viva

1. Import libraries
 2. Load Fashion-MNIST dataset
 3. Display sample images
 4. Reshape images for CNN
 5. Normalize pixel values
 6. Apply one-hot encoding to labels
 7. Build CNN model
 8. Add convolution and pooling layers
 9. Flatten feature maps
 10. Add dense output layer
 11. Compile model using categorical crossentropy
 12. Train CNN model
 13. Evaluate accuracy on test data
-

Most Important Viva Questions

1. Why CNN is preferred for images?

Because CNN automatically extracts spatial image features efficiently.

2. What is convolution?

Operation used to extract image features using filters.

3. What is pooling?

Reducing image dimensions while preserving important features.

4. What is flattening?

Converting 2D feature maps into 1D vector.

5. Why normalize pixels?

To improve learning speed and stability.

6. Why categorical crossentropy?

Because there are multiple output classes.

7. Difference between CNN and ANN?

CNN	ANN
Best for images	General-purpose
Feature extraction automatic	Manual features often needed
Parameter sharing	Large parameters

8. What are filters/kernels?

Small matrices used to scan images and detect patterns.

9. Why pooling reduces overfitting?

Because it reduces unnecessary information and parameters.

10. What is feature extraction?

Automatically detecting important visual patterns from images.